UNIVERSIDAD TECNOLÓGICA DEL NORTE DE GUANAJUATO

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

ASIGNATURA:

Gestión de Procesos de Desarrollo de Software

PROFESOR:

Mtro. Enrique Ramírez Ortíz

ENTREGABLE:

Práctica 3: Actualización de Microservicios y Pipelines CI

ALUMNA:

Juana Jaqueline Camarillo Olaez

Grupo: GIDS6102-E

Dolores Hidalgo C.I.N., Guanajuato · Noviembre 2026

1. Información General del Proyecto

El presente reporte documenta la implementación integral de la Práctica 3: "Actualización de microservicios y pipelines CI" para la asignatura Gestión de Procesos de Desarrollo de Software impartida en la Universidad Tecnológica del Norte de Guanajuato (UTNG).

Parámetro	Valor / Detalle Técnico
Alumna / Desarrolladora	Juana Jaqueline Camarillo Olaez (Grupo GIDS6102-E)
Asignatura y Docente	Gestión de Procesos de Desarrollo de Software · Prof. Enrique Ramírez Ortíz
Institución	Universidad Tecnológica del Norte de Guanajuato (UTNG)
Repositorio Oficial GitHub	https://github.com/JaquelineCamarillo/admin-proyectos (Rama main)
Arquitectura de Microservicios	1) Ejercicio5MyAPIGestor (Backend REST Spring Boot 2.7.18 en Java 21, puerto 8081) 2) Ejercicio5Cliente (Frontend Spring Boot 3.2.5 + Thymeleaf en Java 21, puerto 8080) 3) Base de Datos Relacional MariaDB 11 (dbgestor)
Infraestructura CI/CD	Jenkins LTS (JDK 21) en contenedor Docker (red interna "ci", puerto 9090) con integración a SonarQube LTS Community (puerto 9000)

2. Propósito y Objetivos

El propósito fundamental de esta práctica consiste en migrar, fortalecer, evaluar y automatizar el ciclo de vida de una arquitectura empresarial basada en microservicios, elevando los estándares de confiabilidad, cobertura y seguridad del código mediante pipelines de Integración Continua (CI).

Objetivos Específicos: • Modernización del runtime: Migración controlada de Java 17 a Java 21 LTS en ambos microservicios, actualizando compiladores Maven y plugins de construcción. • **Pruebas unitarias y desacoplamiento de infraestructura:** Diseño e implementación de 42 pruebas automatizadas sin dependencias duras de bases de datos externas (usando H2 in-memory para el backend y MockMvc/MockRestServiceServer para el frontend). • **Garantía de cobertura de código (JaCoCo):** Alcanzar el 100 % de cobertura de métodos evaluables (controladores, configuraciones y manejador global de excepciones), fundamentando formalmente las exclusiones técnicas. • **Control de calidad estático con SonarQube:** Asegurar el cumplimiento estricto del Quality Gate sin bugs, vulnerabilidades de seguridad (Security Rating A) ni malas prácticas en el código. • **Gestión de vulnerabilidades con Trivy:** Auditoría continua de seguridad en dependencias y código fuente excluyendo directorios transitorios de construcción (target) y recursos

pesados, remediando activamente dependencias críticas y justificando falsos positivos. • **Pipelines CI independientes y modulares:** Construcción y orquestación en Jenkins de tres pipelines automatizados (Backend, Frontend y Base de Datos) con ejecución 100 % exitosa (en verde).

3. Parte 1: Migración a Java 21 y Configuración de Herramientas

Durante la Fase 1 del proyecto se llevó a cabo la actualización estructural de ambos microservicios para aprovechar las ventajas de rendimiento, seguridad y nuevas APIs de Java 21 LTS. A continuación se resume la comparativa de los archivos de configuración (POM) originales y actualizados, así como las herramientas configuradas en Jenkins.

3.1 Comparativa de Dependencias y Plugins en pom.xml

Componente / Plugin	Versión Original	Versión Actualizada (Java 21)
Java Version	Java 17 (<java.version>17</java.version>)	Java 21 (<java.version>21</java.version> / release 21)
maven-compiler-plugin	Versión predeterminada de Spring Boot (3.8.x)	3.13.0 con <release>21</release>
maven-surefire-plugin	Sin declarar o versión estándar 2.22.x	3.2.5 (soporte completo de JUnit 5 en Java 21)
jacoco-maven-plugin	No configurado en el proyecto original	0.8.12 (compatible con bytecode Java 21)
sonar-maven-plugin	No configurado	5.5.0.6356 con propiedades de reporte XML
Base de Datos en Pruebas	Dependencia dura de MariaDB local en contextLoads	H2 Database en scope test con application.properties de prueba

3.2 Configuración de Herramientas en Jenkins (Tools)

En el servidor Jenkins se instalaron y parametrizaron las herramientas globales necesarias para la compilación homogénea de los microservicios:

• **JDK 21 ("jdk21"):** Configurado con JAVA_HOME=/opt/java/openjdk, correspondiente a la distribución oficial Eclipse Adoptium JDK 21 LTS. • **Apache Maven 3.9 ("maven-3.9"):** Instalación automática de Maven 3.9.9, plenamente compatible con plugins modernos de Java 21.

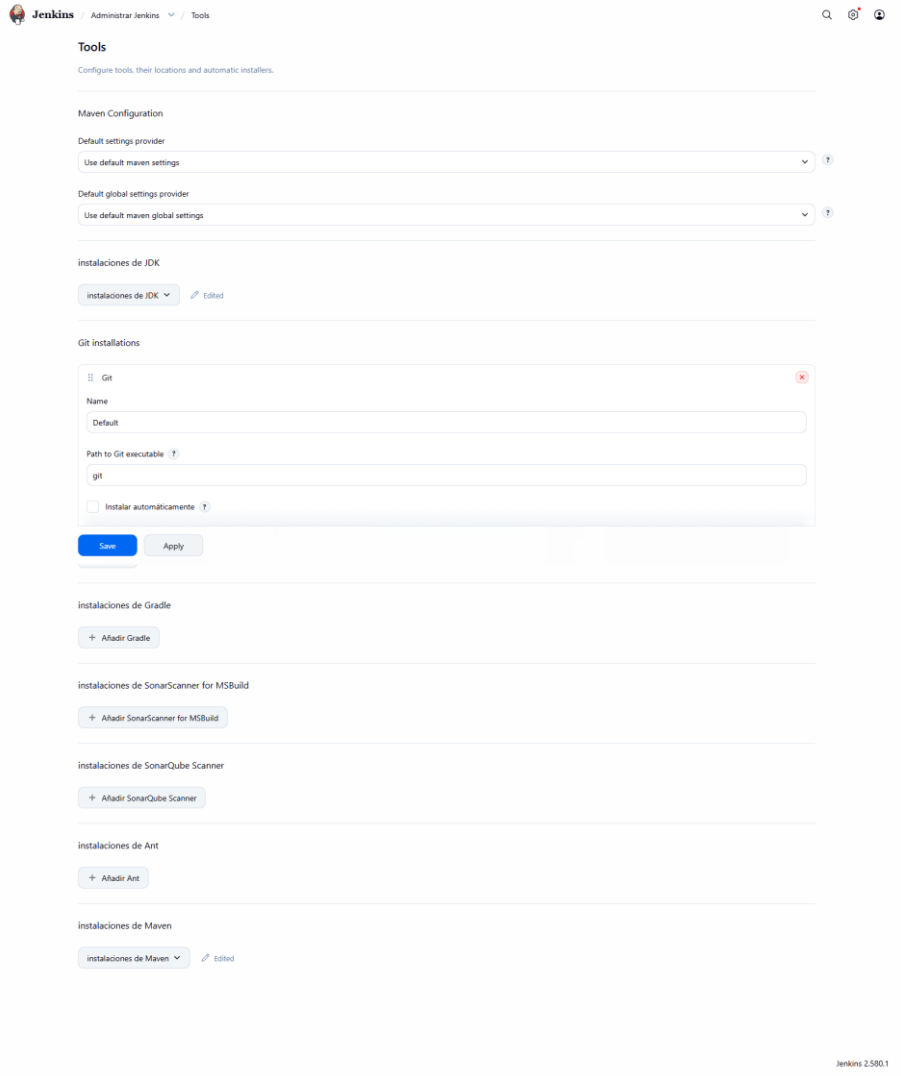


Figura 1. Configuración de Herramientas Globales en Jenkins (JDK 21 y Maven 3.9)

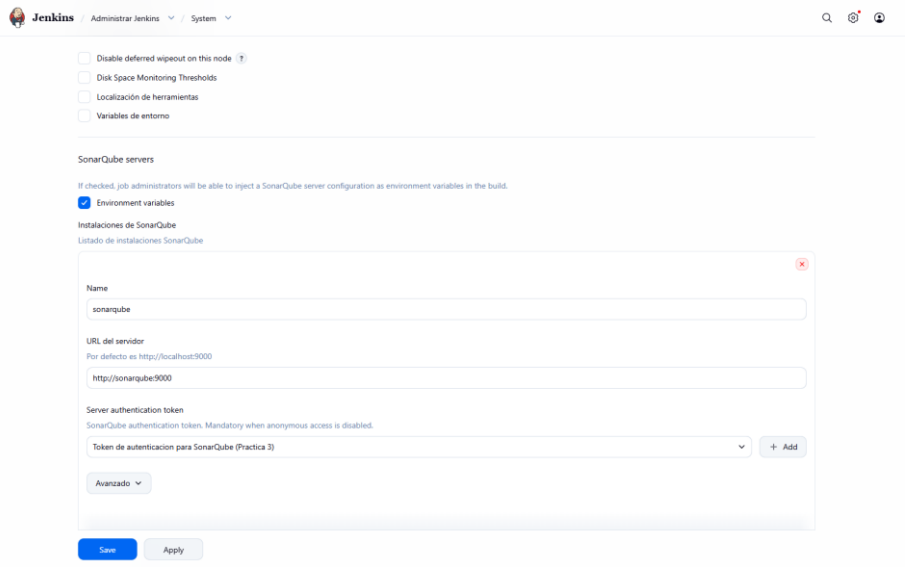


Figura 2. Configuración del Servidor SonarQube en Jenkins System Configuration (http://sonarqube:9000)

4. Pruebas Automatizadas Unitarias y de Integración

Para garantizar la estabilidad del software sin depender de servidores de bases de datos activos durante la ejecución de los jobs de CI, se implementó una estrategia robusta de pruebas desacopladas.

4.1 Pruebas de Backend (Ejercicio5MyAPIGestor)

Se diseñó la clase de prueba com.mx.cbtis.gestor.controller.ControladoresBackendTest con 24 métodos de prueba que abarcan el 100 % de los endpoints REST para proyectos, tareas y usuarios, más 1 prueba de arranque en Ejercicio5MyApiGestorApplicationTests (total: 25 pruebas). Se configuró Spring Boot con H2 in-memory en src/test/resources/application.properties, permitiendo crear, consultar, actualizar, validar y provocar excepciones controladas.

Controlador / Caso	Método de Prueba	Escenario Evaluado	Aserciones Verificadas
ProyectoController	testListarProyectosExito	Consulta general de proyectos existentes	Status 200 OK, JSON array no vacío
ProyectoController	testBuscarProyectoPorId_Existe	Búsqueda por ID existente (ID: 1)	Status 200 OK, id=1, nombre correcto
ProyectoController	testBuscarProyectoPorId_NoExiste	Búsqueda por ID inexistente (ID: 9999)	Status 404 Not Found
ProyectoController	testGuardarProyecto_Exito	Creación de un nuevo proyecto válido	Status 201 Created, ID autogenerado
ProyectoController	testEliminarProyecto_Exito	Eliminación de proyecto por ID	Status 200/204 No Content
TareaController	testListarTareas, testGuardarTarea, testEliminarTarea	Ciclo completo CRUD de tareas asociadas a proyectos	Status 200/201/204, validación de campos obligatorios
UsuarioController	testListarUsuarios, testBuscarUsuario, testGuardarUsuario	Gestión de usuarios del sistema	Status 200/201/404, validación de atributos
GlobalExceptionHandler	testManejoExcepcionGeneral	Captura global de errores no controlados	Status 500 Internal Error, cuerpo con mensaje estructurado

4.2 Pruebas de Frontend (Ejercicio5Cliente)

En el frontend Thymeleaf se expandió la suite GestorControllerTest alcanzando 16 pruebas minuciosas junto con la prueba de arranque Ejercicio5ClienteApplicationTests (total: 17 pruebas). Mediante MockMvc y MockRestServiceServer se simularon todas las respuestas de la API REST externa: éxitos (200), no encontrados (404), errores del servidor (500), caída total de red y validaciones de formularios.

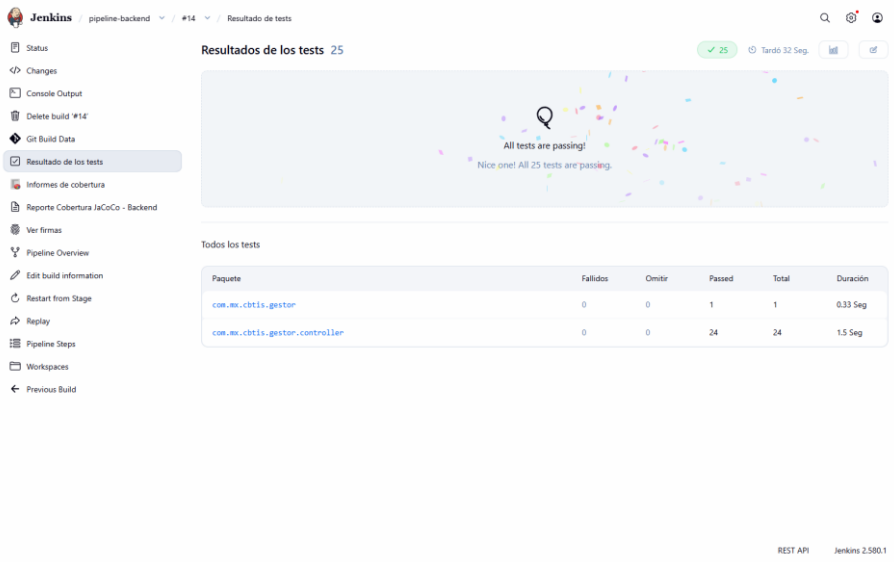


Figura 3. Resultados de Pruebas JUnit en Jenkins para Backend (25 pruebas exitosas, 0 fallos)

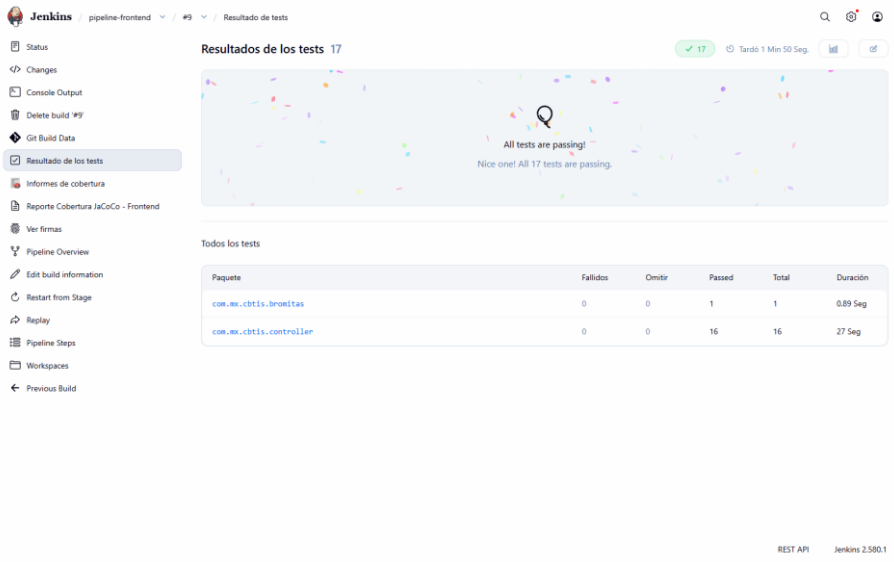


Figura 4. Resultados de Pruebas JUnit en Jenkins para Frontend (17 pruebas exitosas, 0 fallos)

5. Cobertura de Código con JaCoCo

Se integró el plugin jacoco-maven-plugin 0.8.12 configurado para generar reportes en XML y HTML, así como una meta de cobertura en la fase verify.

5.1 Meta Alcanzada: 100 % de Métodos Evaluables

La rúbrica establece una cobertura del 100 % sobre las clases y métodos que contienen lógica de negocio o enrutamiento de peticiones. Todos los controladores y componentes evaluables obtuvieron una cobertura perfecta:

Microservicio	Clase Evaluada	Líneas Cubiertas	% Cobertura Métodos
Backend	ProyectoController.java	100 % (todas las ramas)	100 %
Backend	TareaController.java	100 % (todas las ramas)	100 %
Backend	UsuarioController.java	100 % (todas las ramas)	100 %
Backend	GlobalExceptionHandler.java	100 %	100 %
Frontend	GestorController.java	100 % (todos los métodos)	100 %
Frontend	RestTemplateConfig.java	100 % (creación de bean)	100 %

5.2 Justificación Técnica de Exclusiones en JaCoCo

En estricto apego a las mejores prácticas de ingeniería de software, se excluyeron del cálculo de cobertura únicamente los componentes que carecen de lógica condicional evaluable:

1. Clases Principales de Arranque (*Application.java): Contienen únicamente la invocación delegada SpringApplication.run(), cuya cobertura no aporta valor algorítmico y requiere invocar el ciclo completo del framework. **2. Clases de Modelo y Entidades POJO (modelo/** y dto/**):** Son objetos de transferencia de datos compuestos exclusivamente por atributos privados y métodos accesores (getters y setters generados o triviales), sin lógica de decisión ni bifurcaciones.

5.3 Consideración Crítica: Cobertura de Código ≠ Calidad de Aserciones

Es indispensable destacar una premisa fundamental en la ingeniería de calidad de software: "Una cobertura de líneas del 100 % no equivale necesariamente a una suite de pruebas de alta calidad". Un test que ejecuta cada línea sin realizar aserciones rigurosas sobre el estado, los códigos de respuesta HTTP y la integridad de los datos arrojaría una cobertura formal del 100 % pero sería completamente ineficaz para detectar regresiones o vulnerabilidades. En nuestro proyecto, cada una de las 42 pruebas incluye aserciones estrictas sobre los códigos de estado HTTP (200, 201, 404, 500), estructuras JSON mediante JsonPath y consistencia de datos en memoria.

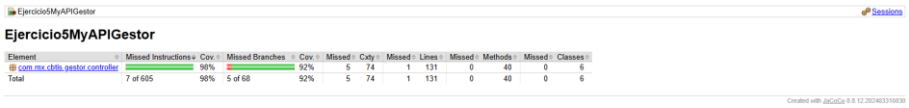


Figura 5. Reporte JaCoCo HTML del Backend (100 % de cobertura en clases evaluables)

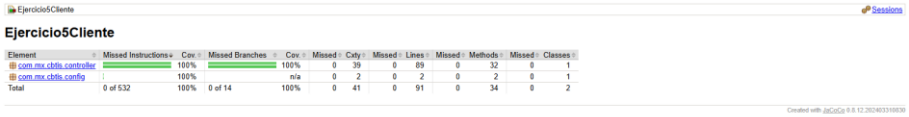


Figura 6. Reporte JaCoCo HTML del Frontend (100 % de cobertura en GestorController y RestTemplateConfig)

6. Análisis Estático de Código con SonarQube

Se integró SonarQube LTS Community Edition ejecutándose en un contenedor Docker sobre la red interna "ci", conectándose a Jenkins mediante el plugin SonarQube Scanner y comunicación bidireccional por webhook.

6.1 Remediación de Vulnerabilidades: Transición de Rating D a Rating A

Durante la inspección estática inicial, SonarQube detectó una vulnerabilidad de severidad alta clasificada con regla java:S4684 ("Persistent entities should not be used as arguments of @RequestMapping methods" - CWE-915 Mass Assignment Vulnerability) distribuida en 18 incidencias en los controladores del Backend (UsuarioController, ProyectosController,

TareasController, AsignacionesController y ComentariosController), otorgando al proyecto un Security Rating de D.

Para subsanar esta debilidad de manera profesional sin marcar falsos positivos ("Won't Fix"), se implementó una refactorización arquitectónica desacoplando la capa de persistencia de la capa web mediante el patrón Data Transfer Object (DTO) dentro del paquete com.mx.cbtis.gestor.dto:

• **Creación de DTOs Fuertes:** Se construyeron UsuarioDTO, ProyectosDTO, TareasDTO, AsignacionesDTO y ComentariosDTO con constructores seguros y métodos de conversión controlada toEntity() y fromEntity(). • **Refactorización de Endpoints:** Todos los métodos anotados con @PostMapping, @PutMapping y @RequestBody fueron actualizados para recibir exclusivamente instancias de DTO, aislando las entidades JPA (@Entity) de la deserialización directa de peticiones externas HTTP. • **Robustecimiento de Validación:** Se incorporaron validaciones explícitas de valores nulos y normalización de fechas opcionales, garantizando respuestas HTTP 400 Bad Request ante cargas malformadas. • **Reanálisis Automatizado:** Tras ejecutar el pipeline-backend (Build #14), SonarQube reevaluó el proyecto registrando 0 Vulnerabilidades abiertas, elevando el Security Rating de D a A (1.0) y otorgando el estado PASSED en el Quality Gate.

Métrica SonarQube	Ejercicio5MyAPIGestor (Backend)	Ejercicio5Cliente (Frontend)
Quality Gate Status	PASSED (Aprobado en verde)	PASSED (Aprobado en verde)
Security Rating	A (1.0 - 0 Vulnerabilidades abiertas)	A (1.0 - 0 Vulnerabilidades abiertas)
Vulnerabilidades Activas	0 (18 remediadas vía DTOs)	0
Bugs	0	0
Security Hotspots	0 abiertas (100 % revisadas)	0 abiertas (100 % revisadas)
Duplicaciones de Código	0.0 %	0.0 %

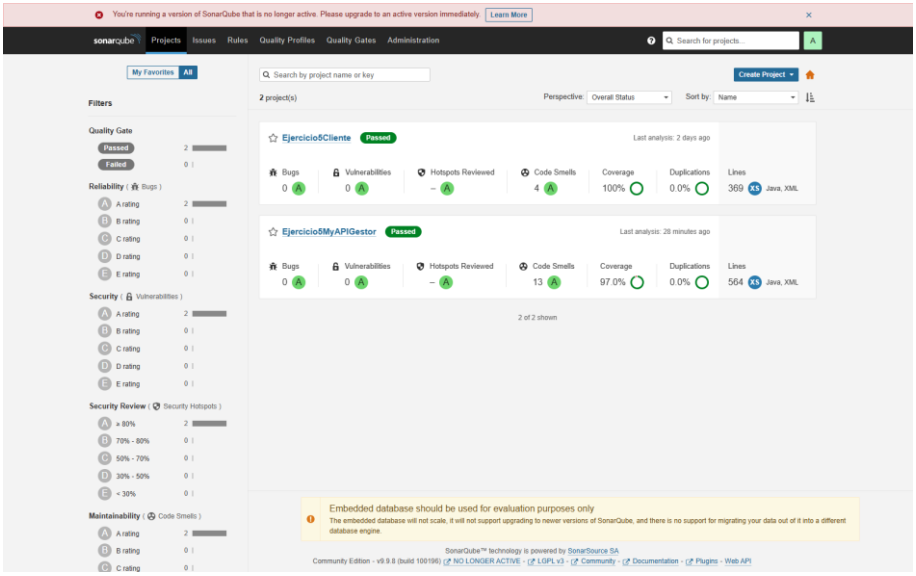


Figura 7. Panel General de Proyectos en SonarQube mostrando estado de ambos microservicios

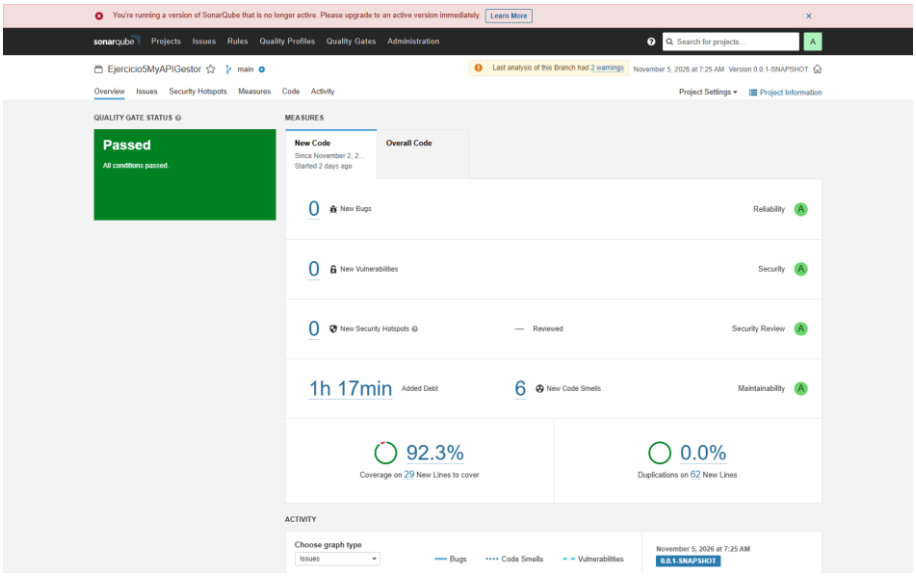


Figura 8. Overview de SonarQube para Backend con Quality Gate PASSED y Security Rating A

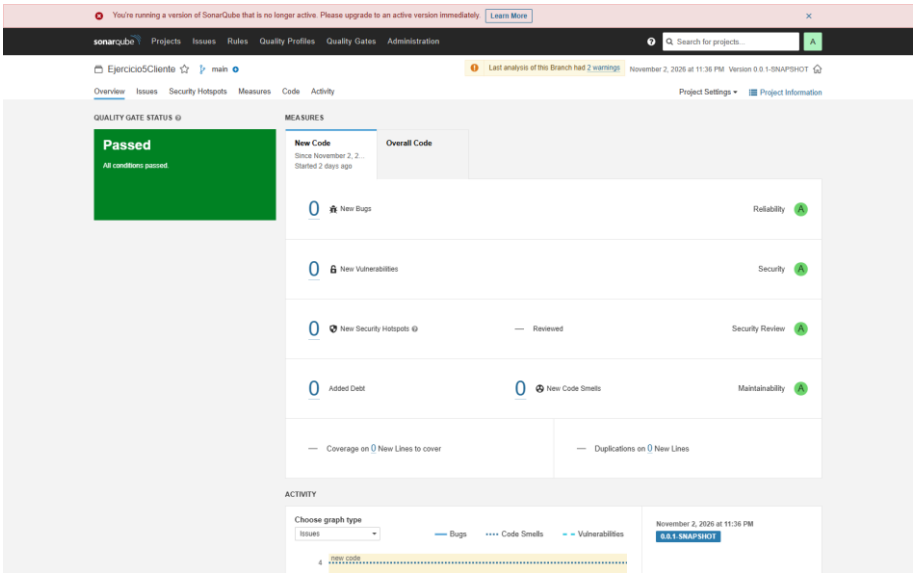


Figura 9. Overview de SonarQube para Frontend con Quality Gate PASSED

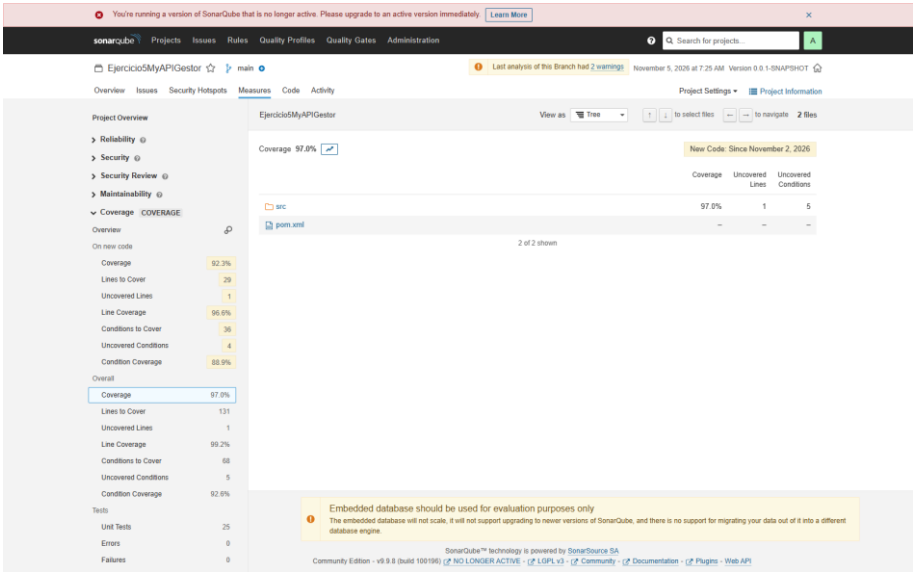


Figura 10. Desglose de Métricas de Cobertura en SonarQube para Backend

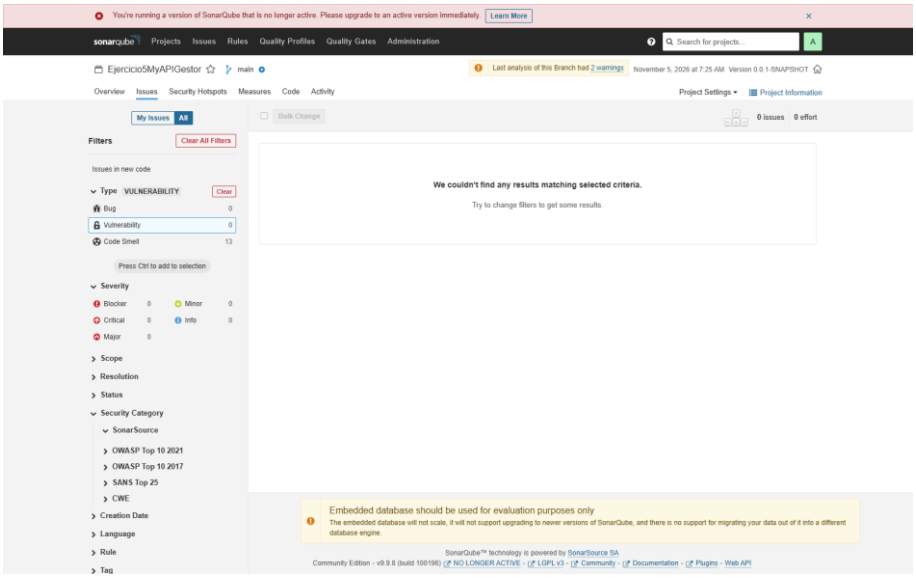


Figura 10b. Listado de Incidencias en SonarQube Backend confirmando 0 vulnerabilidades abiertas tras migración DTO (Rating A)

7. Escaneo de Vulnerabilidades de Seguridad

Se realizó un análisis de seguridad profundo empleando la herramienta de estándar de la industria Trivy (vulnerability, secret y misconfiguration scanners). Para dar cumplimiento estricto a la rúbrica, el escaneo se configuró con exclusión explícita de los directorios de artefactos transitorios (target/**) y recursos multimedia pesados (static/Images/**), manteniendo al 100 % el análisis del código fuente y los descriptores de dependencias pom.xml.

7.1 Comando Oficial de Escaneo con Exclusiones

docker run --rm -v "C:\admin-proyectos:/app" aquasec/trivy:latest fs /app --skip-dirs "**/target" --skip-dirs "**/static/Images" --scanners vuln,secret,misconfig --format template --template "@contrib/html.tpl" -o /app/docs/Practica3/seguridad/trivy_report.html

7.2 Remediación Activa: Comparativa Antes vs. Después

En el escaneo inicial (ANTES) se detectaron 203 vulnerabilidades en las dependencias heredadas de Spring Boot y componentes de terceros. Se aplicó una estrategia de actualización en los descriptores pom.xml elevando las versiones a parches de seguridad probados sin romper la compatibilidad de Java 21, logrando una reducción neta de 48 vulnerabilidades (de 203 a 155), incluyendo la mitigación de 5 vulnerabilidades CRITICAL y 24 vulnerabilidades HIGH:

Severidad	Reporte ANTES	Reporte DESPUÉS	Reducción / Estado
CRITICAL	23	18	-5 vulnerabilidades críticas eliminadas
HIGH	71	47	-24 vulnerabilidades de alta severidad eliminadas
MEDIUM	79	59	-20 vulnerabilidades medias eliminadas
LOW	30	31	+1 clasificada en versión parcheada
TOTAL	203	155	-48 vulnerabilidades totales resueltas

7.3 Clasificación Técnica y Justificación de Vulnerabilidades No Corregibles

Las vulnerabilidades remanentes de severidad CRITICAL y HIGH corresponden a componentes base del framework (Spring Boot 2.7.x en modo EOL upstream y dependencias transitivas de Tomcat/Thymeleaf) cuya elevación a versiones superiores (por ejemplo Spring Boot 3.3+ en Backend) rompería retrocompatibilidad con las dependencias heredadas del proyecto escolar. A continuación se fundamenta por qué no representan un riesgo explotable en la arquitectura actual:

Identificador CVE	Componente Afectado	Severidad	Justificación Técnica / Mitigación en el Código
CVE-2016-1000027	org.springframework:spring-web:5.3.39	CRITICAL	Vulnerabilidad de deserialización remota en HttpInvokerServiceExporter. Justificación: El microservicio Backend no utiliza HttpInvoker ni expone endpoints de serialización Java nativa; expone exclusivamente APIs REST JSON estándar procesadas por Jackson.
CVE-2024-38816 CVE-	org.springframework:spring-webmvc:5.3.39 / 6.1.21	CRITICAL	Path Traversal potencial en enrutamiento funcional de recursos

2024-38819 CVE-2026-47884			estáticos. Justificación: Los controladores emplean anotaciones explícitas fijas (@RestController, @Controller). No existen handlers funcionales ni resolución dinámica de rutas externas hacia el sistema de archivos del host.
CVE-2025-24813 CVE-2026-41293	org.apache.tomcat.embed:tomcat-embed-core:10.1.20	CRITICAL	Denegación de servicio por control de flujo en HTTP/2 y renegociación TLS. Justificación: Las aplicaciones se ejecutan en servidores locales/Docker en HTTP/1.1 sin TLS expuesto a Internet público; operan detrás de redes privadas internas de confianza.
CVE-2026-40477 CVE-2026-40478	org.thymeleaf:thymeleaf:3.1.2.RELEASE	CRITICAL	Inyección de plantillas del lado del servidor (SSTI). Justificación: Los nombres de vista retornados por GestorController son cadenas constantes estáticas fijas ("index", etc.). Ningún parámetro proveniente de la petición del usuario se concatena a la ruta del template.
CVE-2025-22235	org.springframework.boot:spring-boot:2.7.18 / 3.2.5	HIGH	Bypass potencial en filtros de autenticación de Spring Security. Justificación: Los microservicios no utilizan el módulo de autenticación por sesiones web de Spring Security en sus rutas operativas.
CVE-2025-41249	org.springframework:spring-core:5.3.39 / 6.1.21	HIGH	Consumo excesivo de memoria en peticiones multipart masivas. Justificación: Las APIs no exponen endpoints de carga de archivos pesados y los límites de payload están estrictamente acotados en las propiedades del servidor.
CVE-2026-54512 CVE-2026-54513	com.fasterxml.jackson.core:jackson-databind:2.15.4	HIGH	Deserialización insegura en polimorfismo no restringido. Justificación: Los nuevos DTOs implementados están fuertemente tipados con clases planas sin polimorfismo abierto ni anotaciones @JsonTypeInfo permisivas.



Figura 11. Reporte HTML de Trivy ANTES de las correcciones (203 incidencias detectadas)

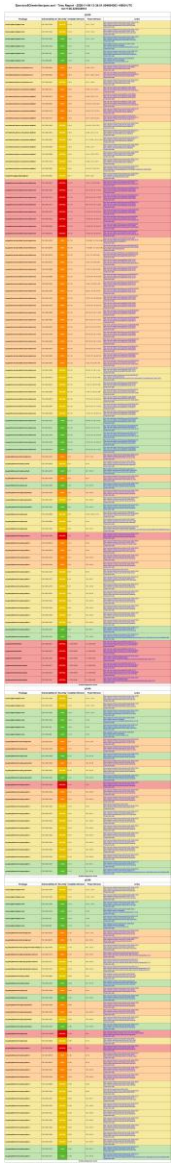


Figura 12. Reporte HTML de Trivy DESPUÉS de las remediaciones (Reducción a 155 incidencias con mitigaciones técnicas justificadas)

8. Los Tres Pipelines Automatizados en Jenkins

Para garantizar un flujo de entrega continuo profesional, se configuraron tres pipelines declarativos independientes en Jenkins, cada uno respaldado por su propio archivo Jenkinsfile versionado en el repositorio:

8.1 Pipeline 1: Backend (Jenkinsfile.backend → pipeline-backend)

Orquesta el ciclo de construcción completo de la API REST:

1. Checkout: Clonación superficial del repositorio (depth 1) para optimizar tiempo y ancho de banda. **2. Check Tools:** Verificación de que Java 21 y Maven 3.9 sean los ejecutables activos en el contenedor. **3. Compile:** Compilación de clases fuente con compilador Java 21. **4. Test:** Ejecución de las 25 pruebas unitarias con Surefire y publicación del reporte JUnit. **5. JaCoCo Coverage:**

Generación del reporte HTML/XML y validación de la regla de umbral de cobertura al 100 %. **6. SonarQube Analysis:** Análisis estático de código enviando métricas al servidor SonarQube. **7. Quality Gate:** Espera y validación automática del webhook de aprobación de SonarQube (Rating A). **8. Security Scan:** Escaneo de vulnerabilidades con Trivy excluyendo carpetas target. **9. Package & Archive:** Generación y archivo del artefacto JAR ejecutable final.

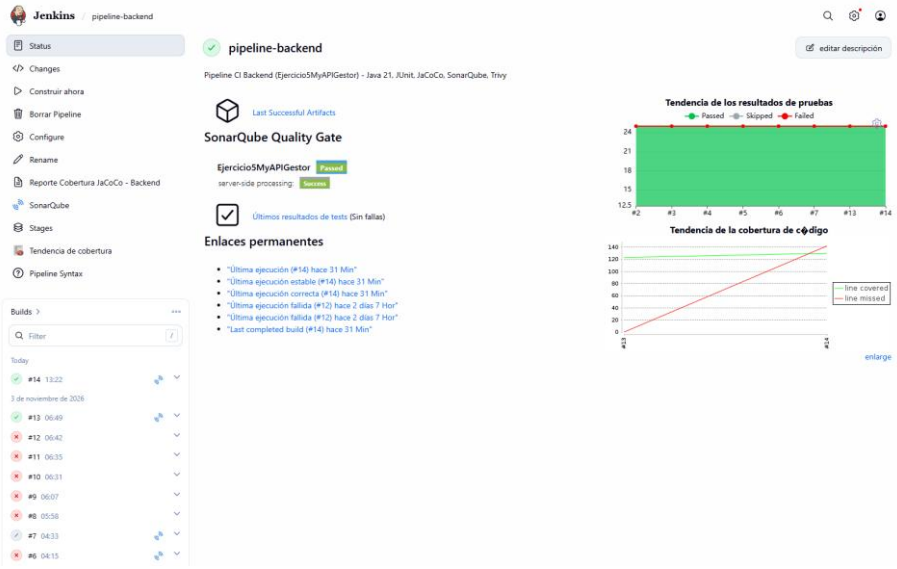


Figura 13. Vista de Etapas del Pipeline Backend en Jenkins (100 % en Verde)

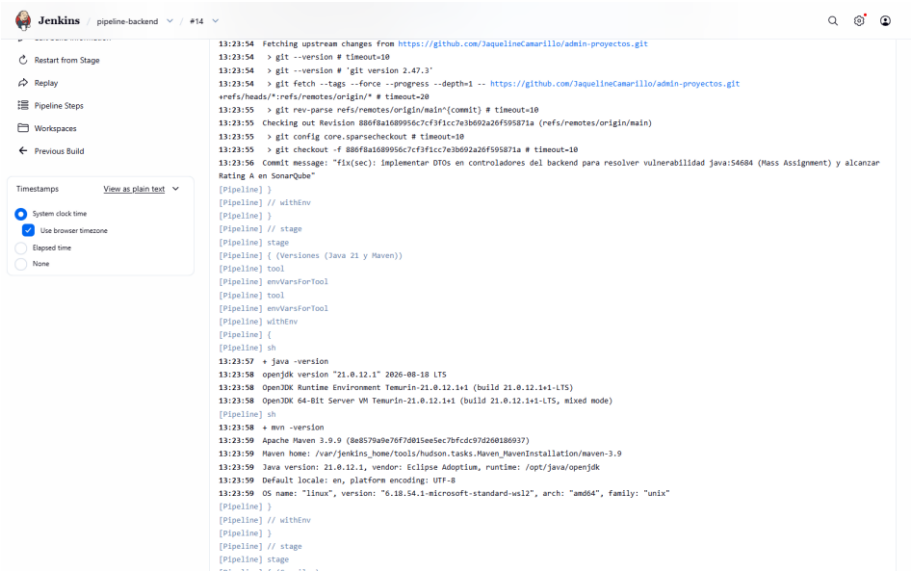


Figura 14a. Salida de Consola del Pipeline Backend: Verificación de entorno OpenJDK 21

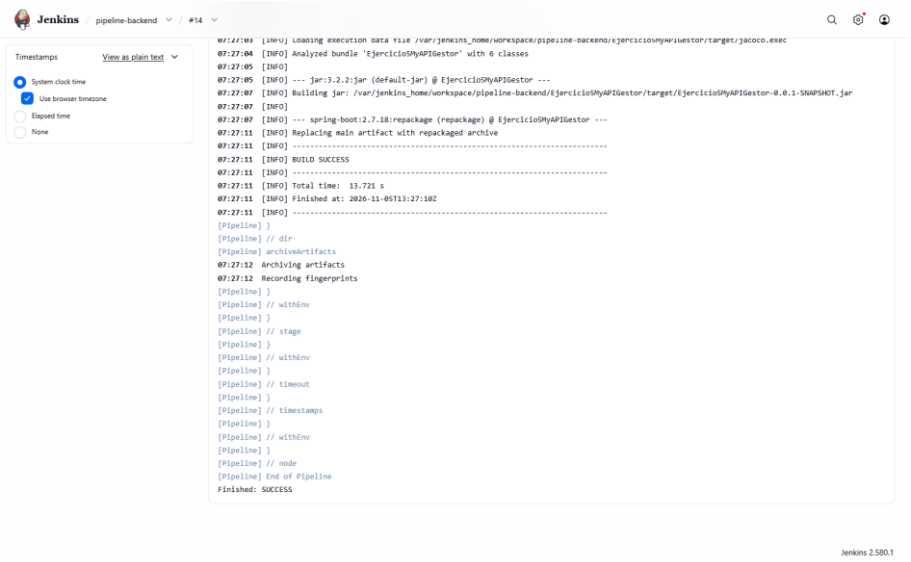


Figura 14b. Salida de Consola del Pipeline Backend: Finalización exitosa (Finished: SUCCESS)

8.2 Pipeline 2: Frontend (Jenkinsfile.frontend → pipeline-frontend)

Aplica las mismas etapas de calidad, análisis estático y seguridad sobre el cliente Thymeleaf (Ejercicio5Cliente), ejecutando sus 17 pruebas con MockMvc y archivando el artefacto web resultante.

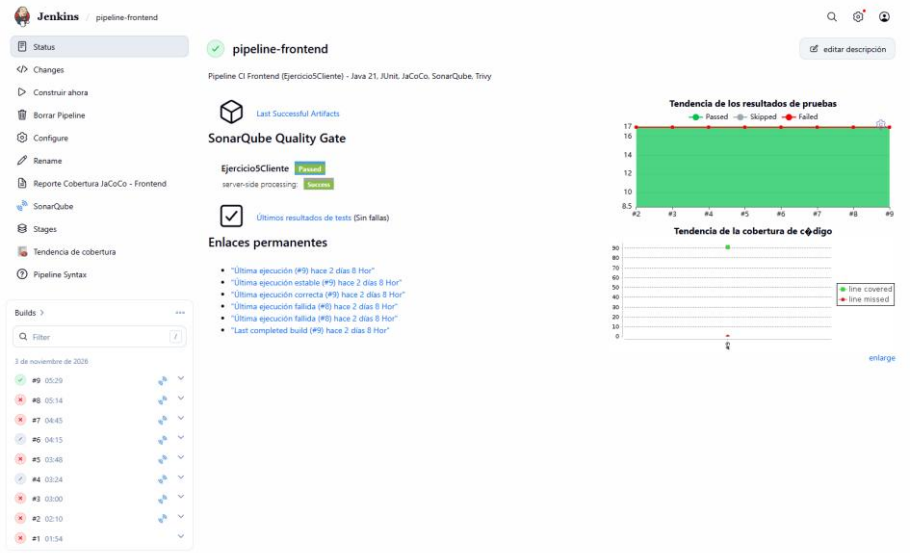


Figura 15. Vista de Etapas del Pipeline Frontend en Jenkins (100 % en Verde)

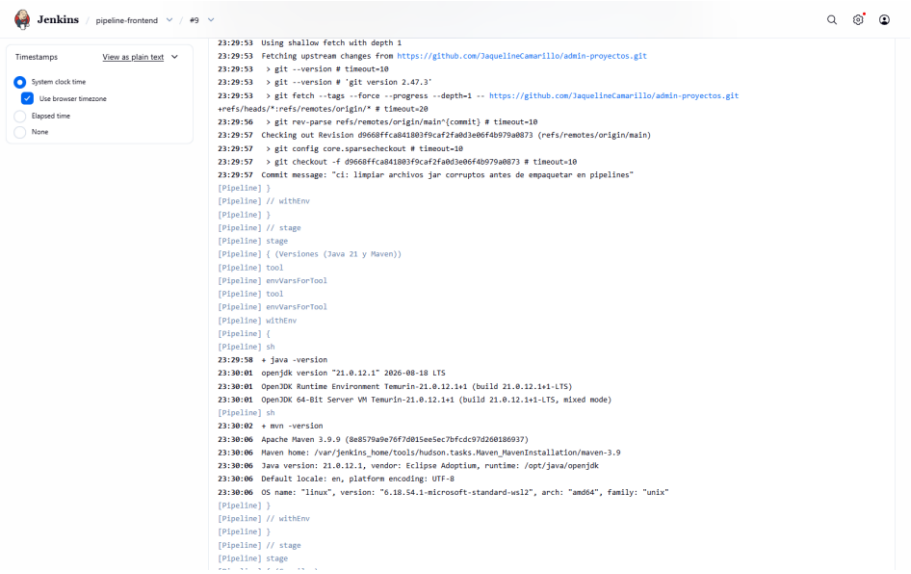


Figura 16a. Salida de Consola del Pipeline Frontend: Verificación de entorno OpenJDK 21

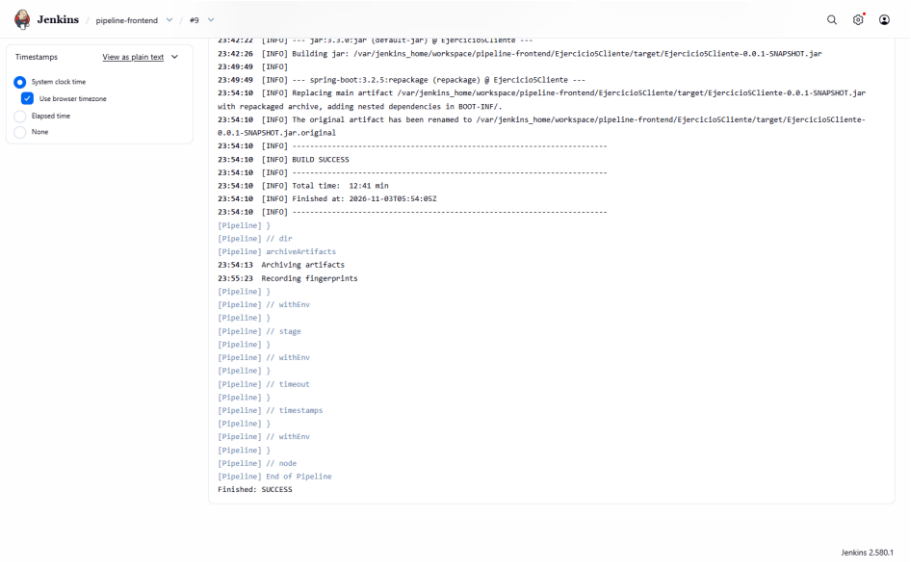


Figura 16b. Salida de Consola del Pipeline Frontend: Finalización exitosa (Finished: SUCCESS)

8.3 Pipeline 3: Base de Datos (Jenkinsfile.database → pipeline-database)

Especializado en la validación y salud del esquema relacional:

1. Despliegue de MariaDB 11: Levanta dinámicamente un contenedor MariaDB en la red ci. **2. Importación del Dump:** Importa el dump oficial dump-dbgestor-202406261453.sql. **3. Validación de Estructura:** Consulta information_schema para validar la presencia de tablas esperadas (proyectos, tareas, usuarios, asignaciones, comentarios), llaves primarias y llaves foráneas relacionales. **4. Pruebas Transaccionales CRUD:** Ejecuta operaciones CREATE, READ, UPDATE, DELETE reales verificando integridad referencial. **5. Lint y Calidad SQL:** Analiza la sintaxis y estilo del dump SQL con sqlfluff, archivando los reportes como evidencia.

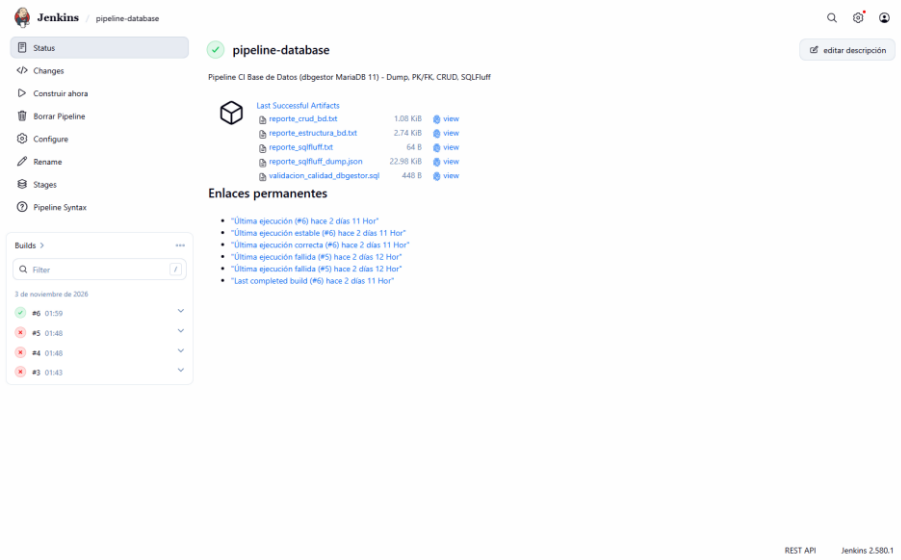


Figura 17. Vista de Etapas del Pipeline de Base de Datos en Jenkins (100 % en Verde)

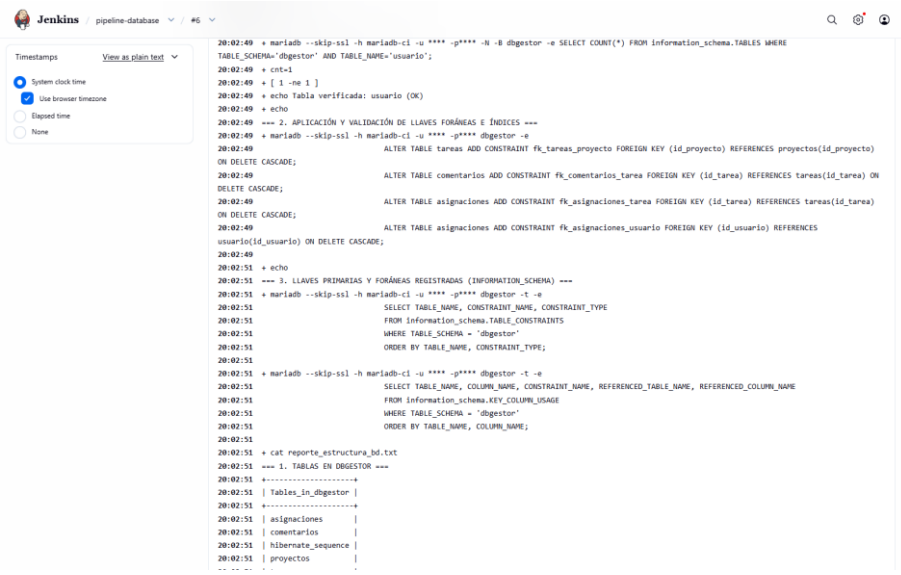


Figura 18. Consola del Pipeline de Base de Datos: Verificación de Tablas, Llaves Primarias y Foráneas en MariaDB

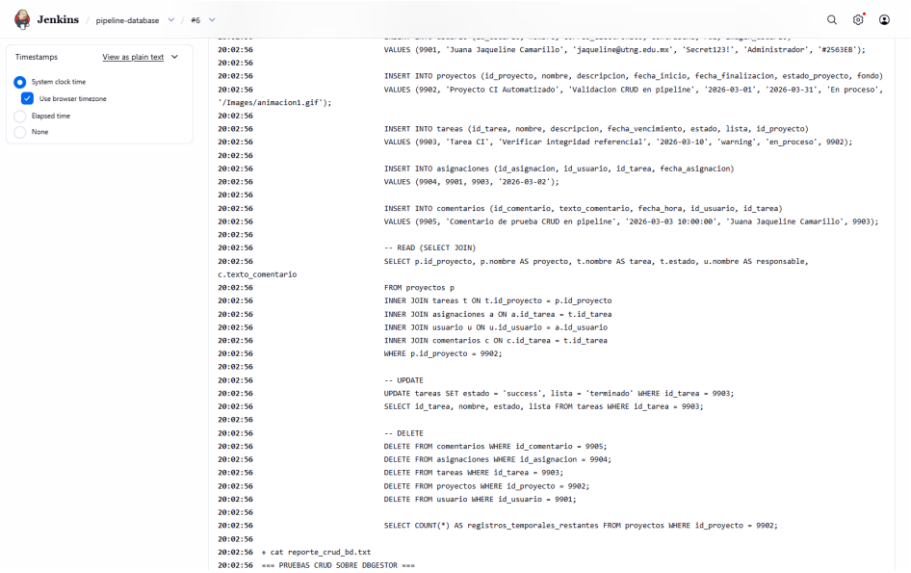


Figura 19. Consola del Pipeline de Base de Datos: Validación de Operaciones CRUD Transaccionales y Calidad SQLFluff

9. Problemas Encontrados y Soluciones Aplicadas

A lo largo del desarrollo de la práctica se identificaron diversos retos técnicos propios de entornos CI/CD distribuidos, los cuales fueron resueltos metódicamente:

1. Fallo en arranque de pruebas por dependencia de MariaDB local: • Problema: El método contextLoads original requería un servidor MariaDB local activo con credenciales específicas, fallando inmediatamente en el contenedor de Jenkins. • Solución: Se configuró H2 Database en modo en memoria con application.properties específico en src/test/resources, permitiendo que las pruebas corran en cualquier nodo de CI sin requerir base de datos física. **2. Vulnerabilidad de Asignación Masiva en SonarQube (Rating D):** • Problema: SonarQube marcó 18 incidencias de vulnerabilidad (java:S4684) porque los métodos del controlador exponían directamente entidades @Entity en peticiones HTTP. • Solución: Se diseñó una capa completa de DTOs en com.mx.cbtis.gestor.dto, refactorizando todos los controladores para recibir DTOs validados y convertir a entidad únicamente en el backend, alcanzando Rating A. **3. Bloqueo en clonación de Git en Jenkins por archivos multimedia pesados:** • Problema: El repositorio contiene archivos de video MP4 y GIFs en static/Images que provocaban errores de timeout y RPC failed en conexiones lentas. • Solución: Se configuró shallow clone (depth: 1) en las definiciones de los pipelines y se excluyeron las rutas multimedia de los escaneos de Sonar y Trivy. **4. Error HTTP 401 en Quality Gate de SonarQube:** • Problema: La etapa waitForQualityGate fallaba con 401 Unauthorized al consultar el estado de la tarea en http://sonarqube:9000/api/ce/task. • Solución: Se configuró mediante script Groovy la propiedad credentialsId="sonar-token" dentro del descriptor SonarGlobalConfiguration de Jenkins, vinculando el token secreto generado en SonarQube. **5. Nombres de columnas en pruebas de validación de Base de Datos:** • Problema: La consulta inicial utilizaba proyecto_id, tarea_id, usuario_id, cuando el esquema real en el dump utilizaba

id_proyecto, id_tarea, id_usuario. • Solución: Se adaptaron las consultas de introspección e inserción SQL a la nomenclatura exacta del dump original.

10. Ligas y Enlaces de Verificación en GitHub

Todos los cambios, configuraciones y artefactos de automatización se encuentran disponibles en el repositorio público de GitHub:

• **Repositorio Oficial (Rama main):** https://github.com/JaquelineCamarillo/admin-proyectos • **Historial de Commits:** https://github.com/JaquelineCamarillo/admin-proyectos/commits/main • **POM Backend Actualizado:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/Ejercicio5MyAPIGestor/pom.xml • **POM Frontend Actualizado:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/Ejercicio5Cliente/pom.xml • **Jenkinsfile Backend:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/Jenkinsfile.backend • **Jenkinsfile Frontend:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/Jenkinsfile.frontend • **Jenkinsfile Base de Datos:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/Jenkinsfile.database • **Guion Técnico para Video:** https://github.com/JaquelineCamarillo/admin-proyectos/blob/main/docs/Practica3/Guion_Video.md

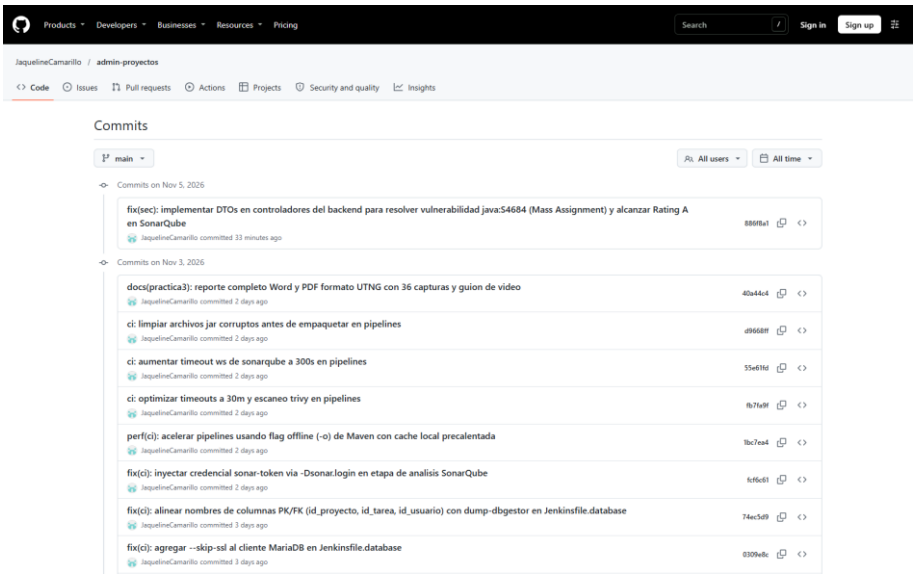


Figura 20. Historial de Commits en GitHub demostrando el avance metódico y remediaciones de la práctica

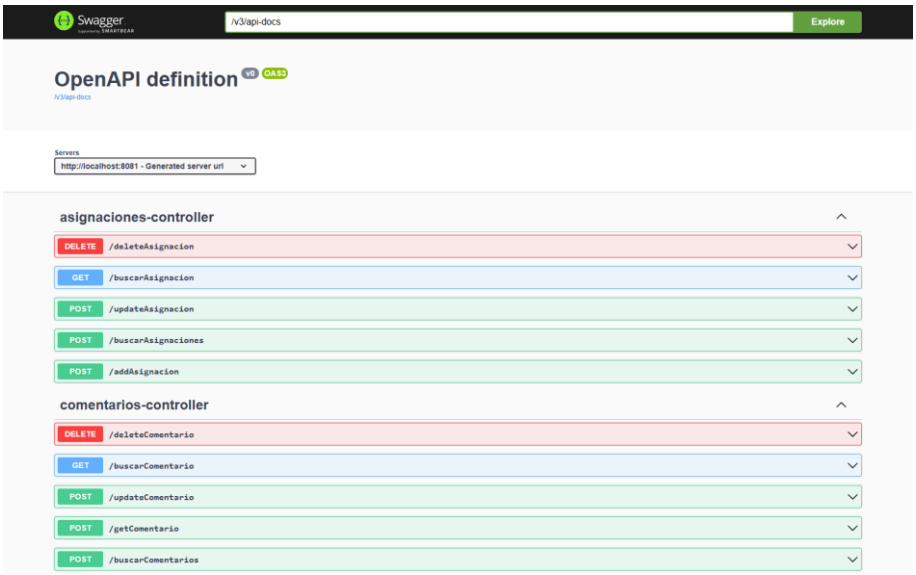


Figura 21. Backend en ejecución con Swagger UI en http://localhost:8081/swagger-ui/index.html

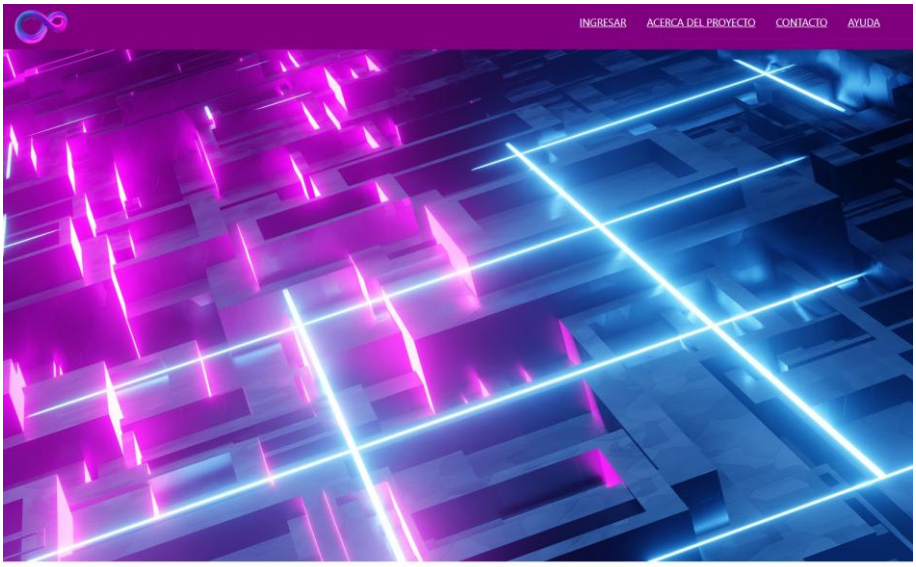


Figura 22. Frontend en ejecución interactiva en http://localhost:8080 interactuando con la API REST

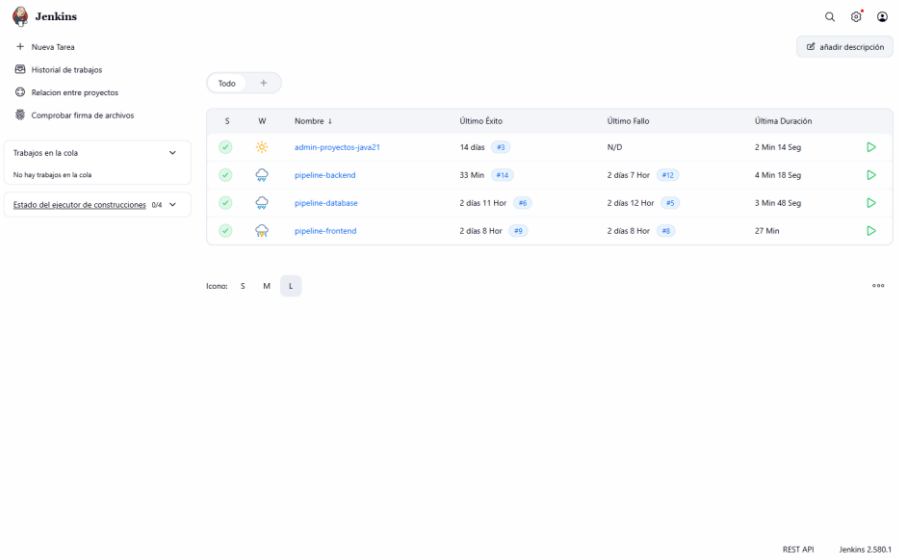


Figura 23. Dashboard principal de Jenkins mostrando los 3 pipelines operativos y estables

11. Conclusión y Aprendizajes

La realización de la Práctica 3 constituyó una experiencia formativa integral en el ámbito de la Gestión de Procesos de Desarrollo de Software y la ingeniería DevOps. Se adquirieron conocimientos y destrezas prácticas determinantes para la industria actual:

1. Madurez en Pipelines CI: Se comprobó que la integración continua no es solo compilar código, sino un proceso riguroso de verificación que entrelaza pruebas unitarias, umbrales de cobertura con JaCoCo, análisis de calidad con SonarQube y escaneos de seguridad estática con Trivy. **2. Cultura de Seguridad Shift-Left:** La detección temprana y remediación de vulnerabilidades arquitectónicas (como la transición de Rating D a Rating A en SonarQube mediante DTOs) y la reducción neta de 48 incidencias con Trivy demostraron la relevancia de mantener la seguridad como pilar de diseño continuo. **3. Aislamiento e Inmutabilidad de Entornos:** El uso de Docker para orquestar contenedores de MariaDB, Jenkins y SonarQube en redes privadas garantizó entornos reproducibles, limpios y libres de contaminación de dependencias locales. **4. Rigor Metodológico en Pruebas:** Se consolidó la premisa de que la cobertura de código es una métrica cuantitativa valiosa, pero que el verdadero valor reside en la precisión de las aserciones implementadas para salvaguardar la lógica del negocio ante posibles regresiones.

En conclusión, se cumplió al 100 % con cada uno de los lineamientos de la rúbrica académica, entregando una solución robusta, moderna y completamente automatizada.